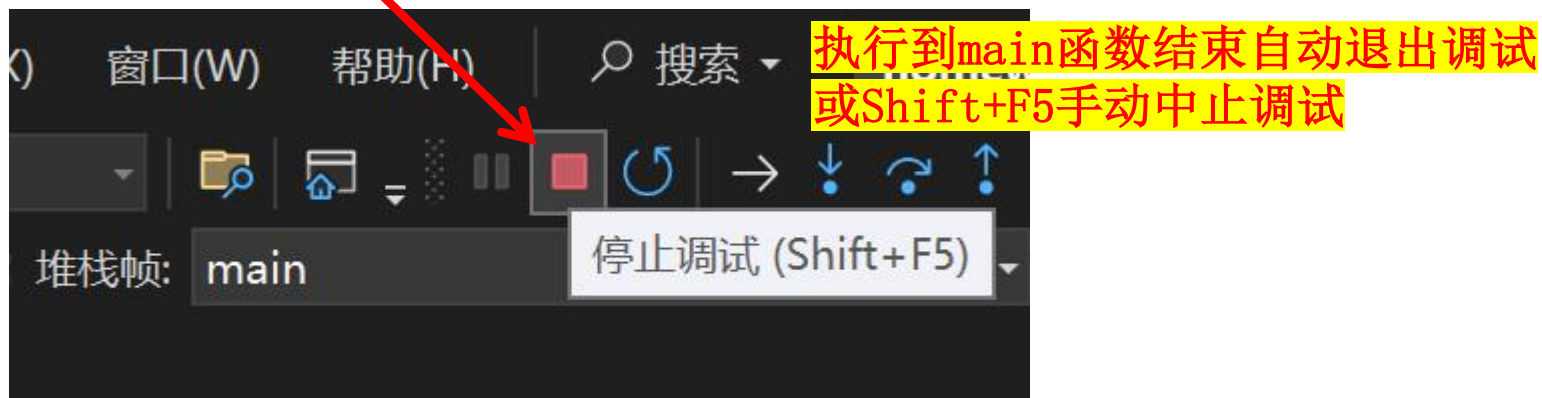
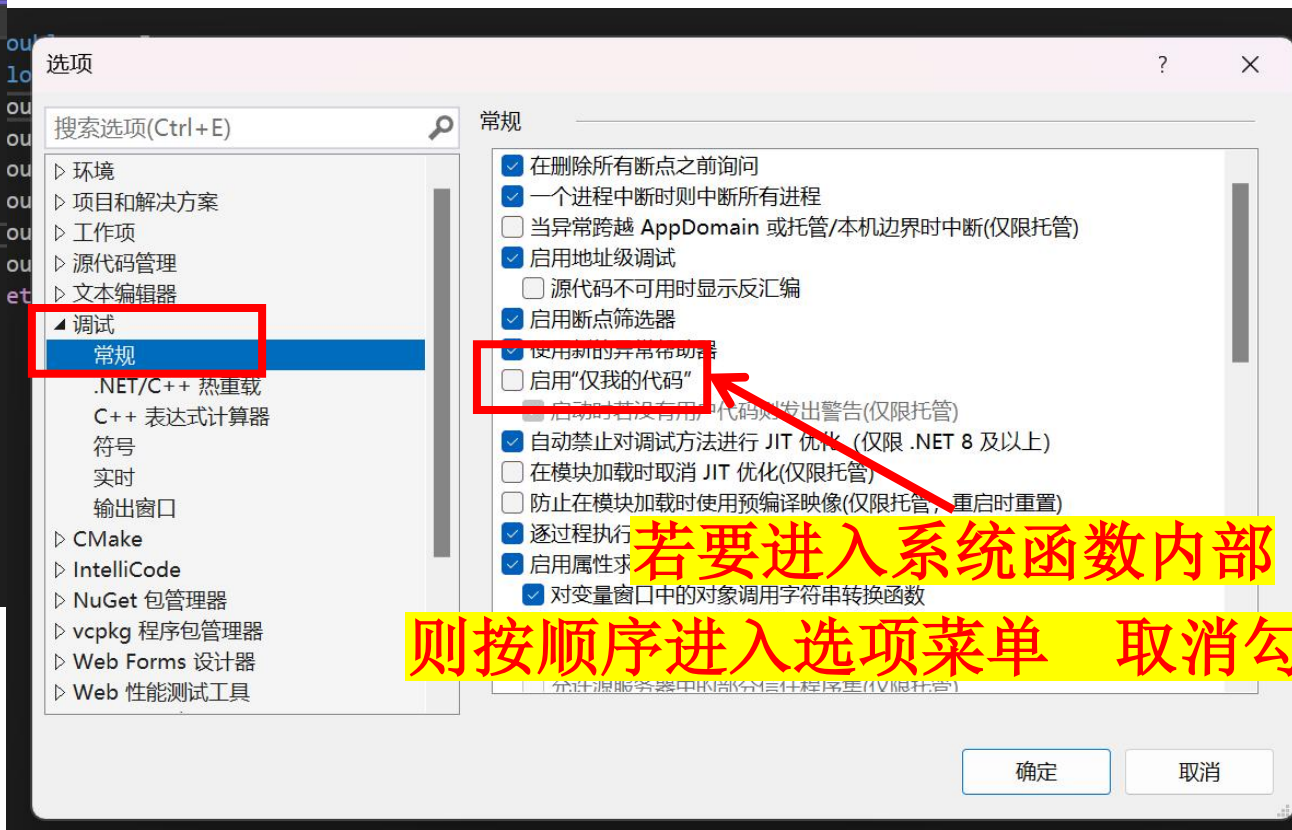
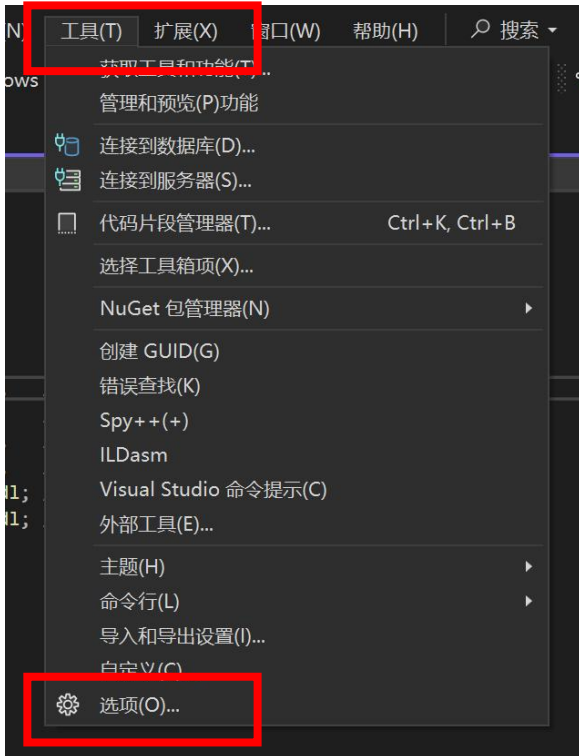
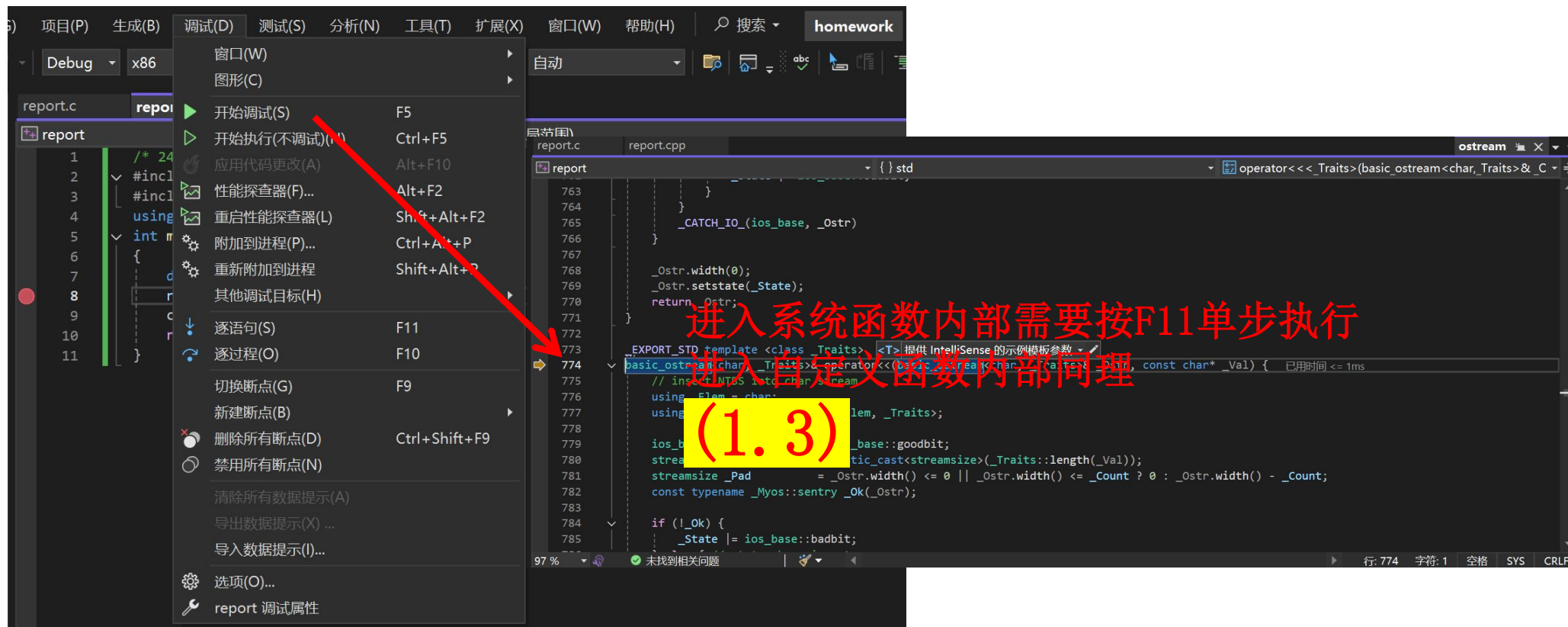
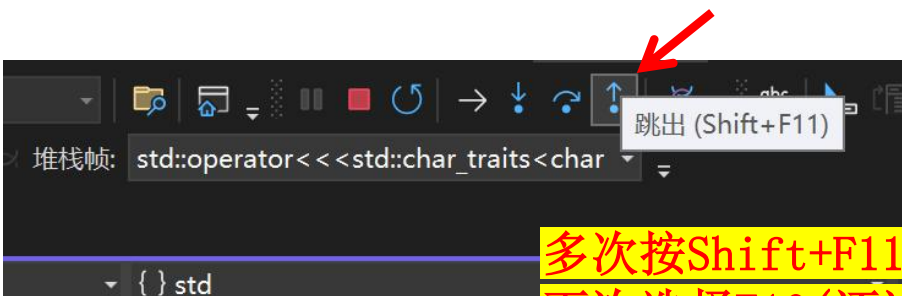


1.1 调试起止

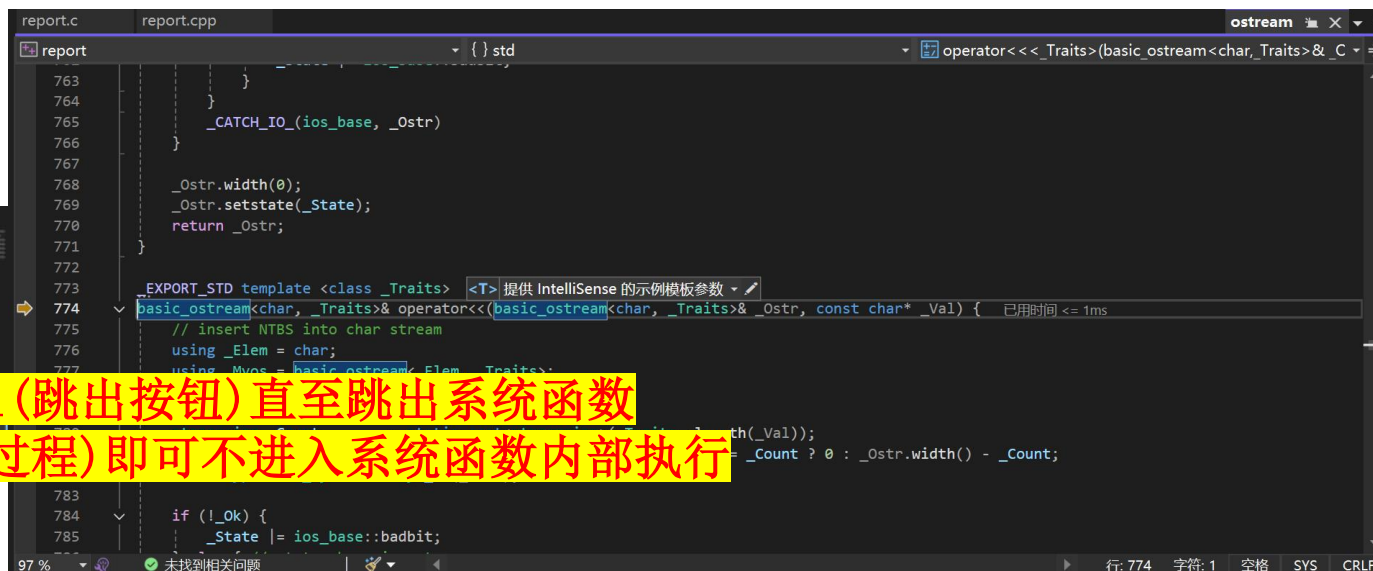


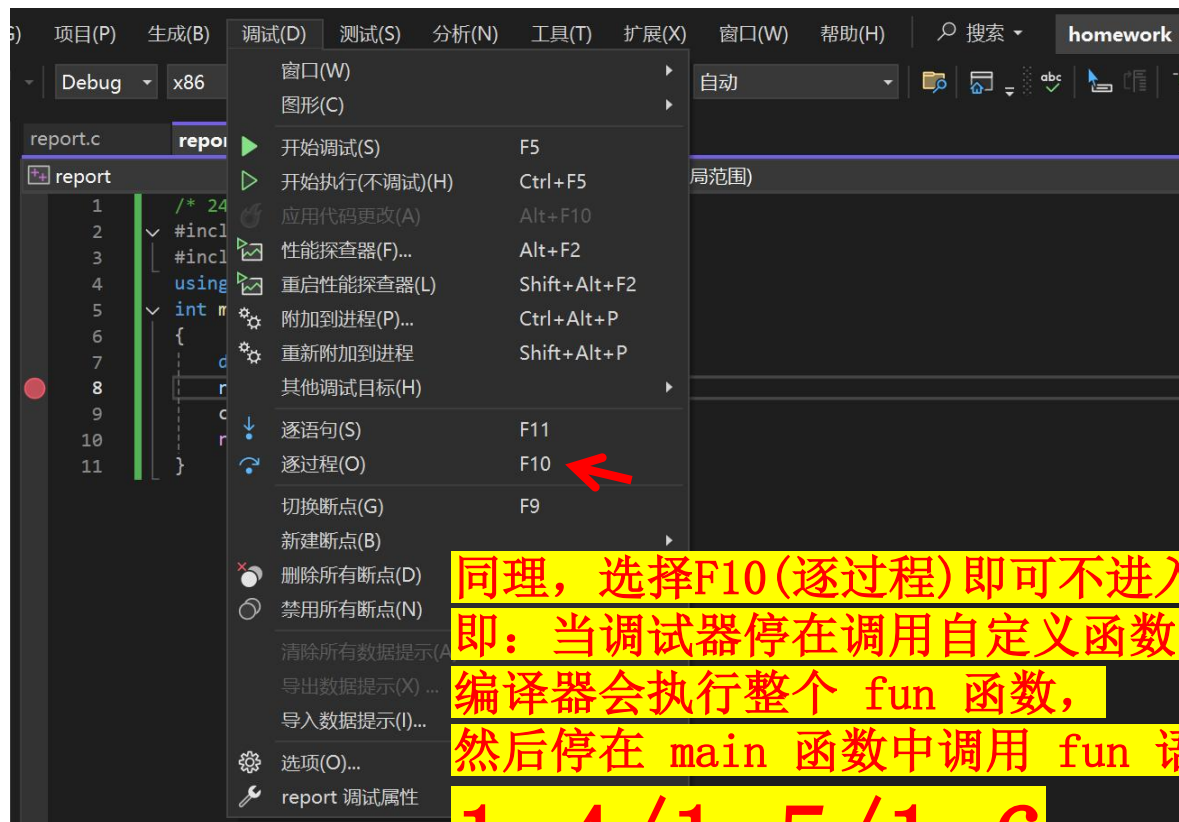






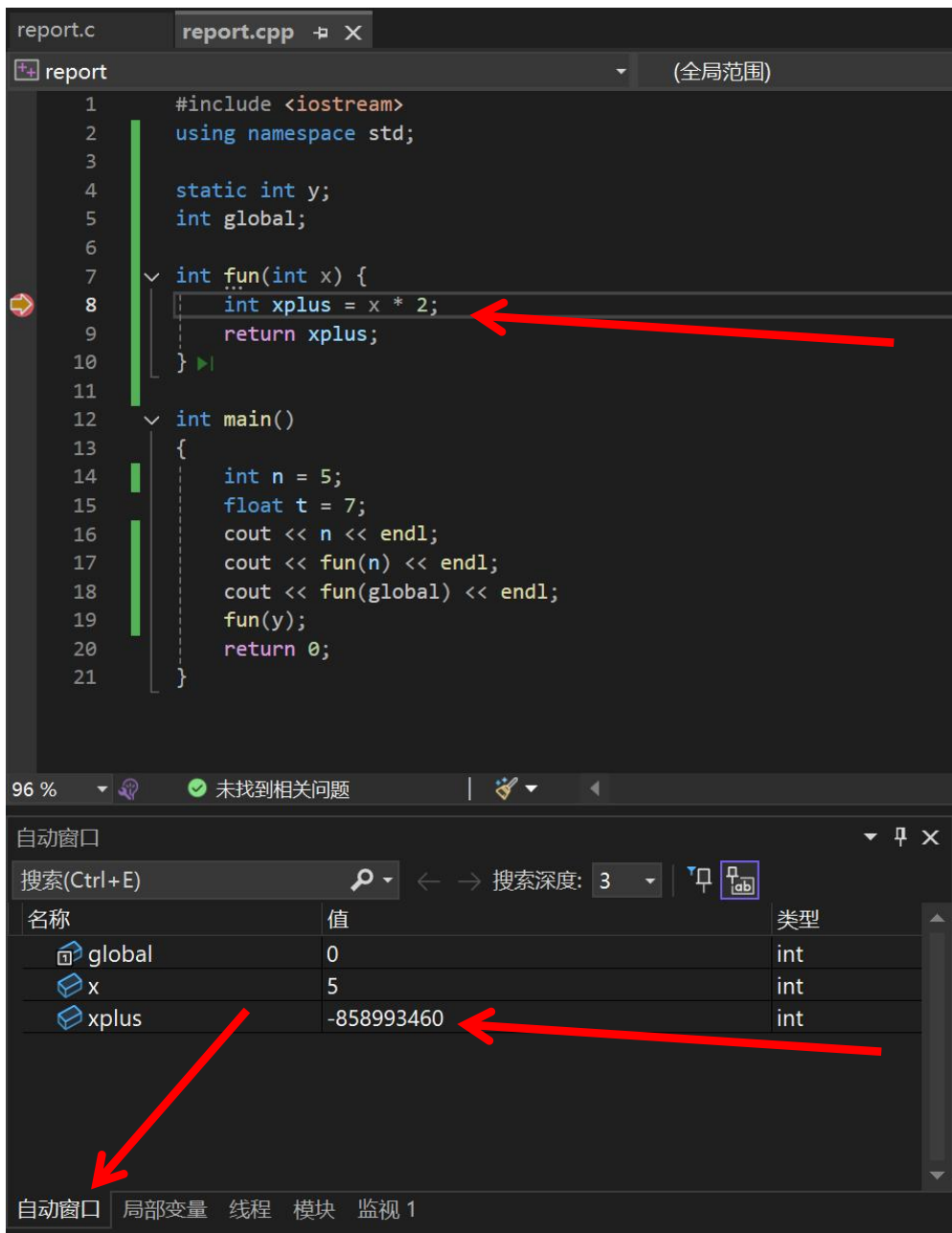
多次按Shift+F11(跳出按钮)直至跳出系统函数
下次选择F10(逐过程)即可不进入系统函数内部执行





同理，选择F10(逐过程)即可不进入自定义函数内部执行
即：当调试器停在调用自定义函数 fun 的那一行代码上时按下F10
编译器会执行整个 fun 函数，
然后停在 main 函数中调用 fun 语句的下一行。

1.4/1.5/1.6



2 查看形参/自动变量的变化情况

当调试停在函数内部的断点处时，自动窗口和局部变量窗口会显示当前函数的形参和所有已声明的自动变量及其值
也可以在监视窗口中手动添加想要观察的形参或自动变量名
通过单步执行（F10 或 F11），观察这些窗口中变量值的变化

形参(x)：在函数开始执行时创建，在函数结束时销亡。

自动变量(xplus)：在函数内部或代码块内部声明的未使用static修饰的普通局部变量

它们在进入其函数 {} 作用域时创建，在退出作用域时销亡。

2.1

```
report.c report.cpp (全局范围)
1 #include <iostream>
2 using namespace std;
3
4 static int y;
5 int global;
6
7 int fun(int x) {
8     int xplus = x * 2;
9     return xplus;
10 }
11
12 int main()
13 {
14     int n = 5;
15     float t = 7;
16     cout << n << endl;
17     cout << fun(n) << endl;
18     cout << fun(global) << endl;
19     fun(y);
20     return 0;
21 }
```

自动窗口

搜索(Ctrl+E) 搜索深度: 3

名称	值	类型
x	5	int
xplus	10	int

自动窗口 局部变量 线程 模块 监视 1

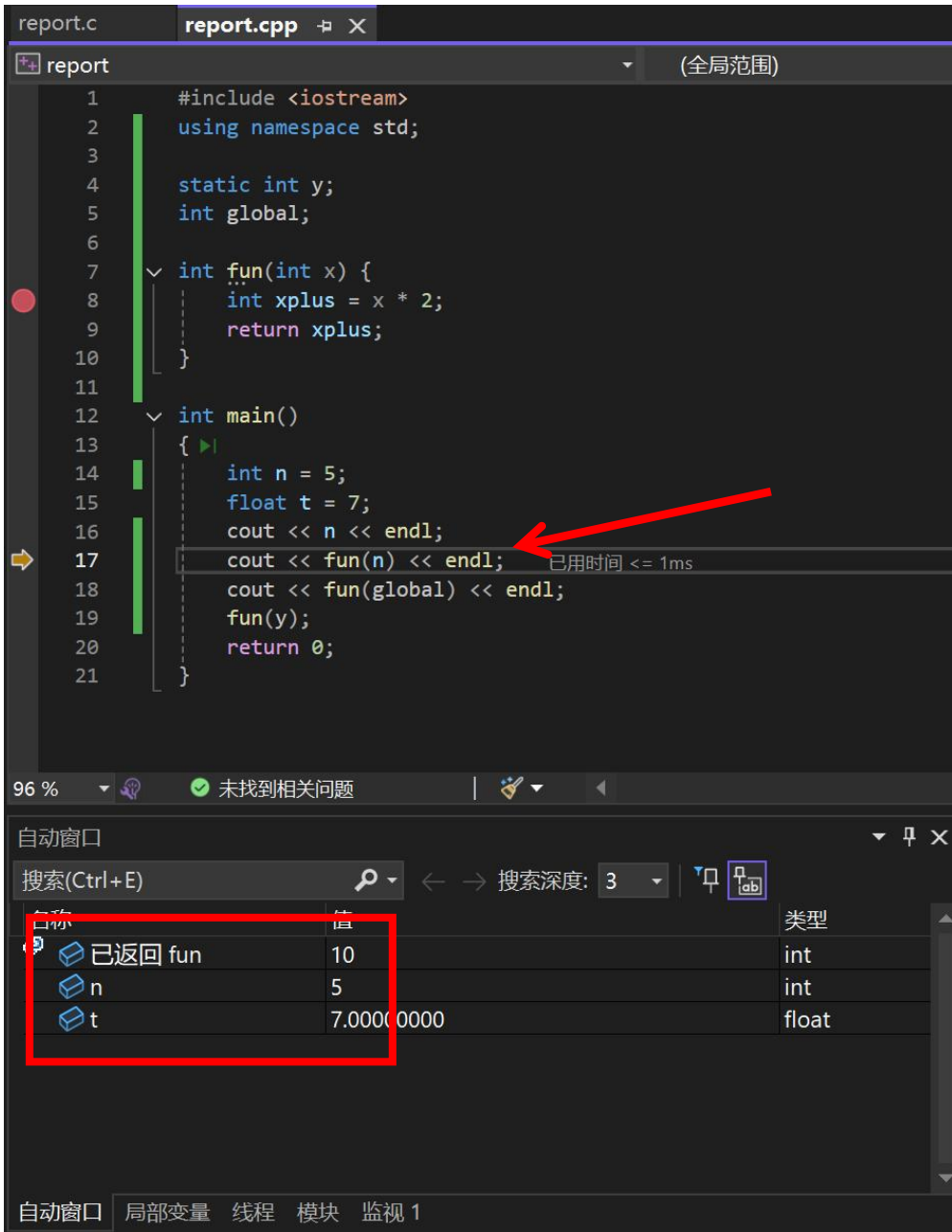
2 查看形参/自动变量的变化情况

当调试停在函数内部的断点处时，自动窗口和局部变量窗口会显示当前函数的形参和所有已声明的自动变量及其值
也可以在监视窗口中手动添加想要观察的形参或自动变量名
通过单步执行（F10 或 F11），观察这些窗口中变量值的变化

形参(x)：在函数开始执行时创建，在函数结束时销亡。

自动变量(xplus)：在函数内部或代码块内部声明的未使用static修饰的普通局部变量

它们在进入其函数 {} 作用域时创建，在退出作用域时销亡。



2 查看形参/自动变量的变化情况

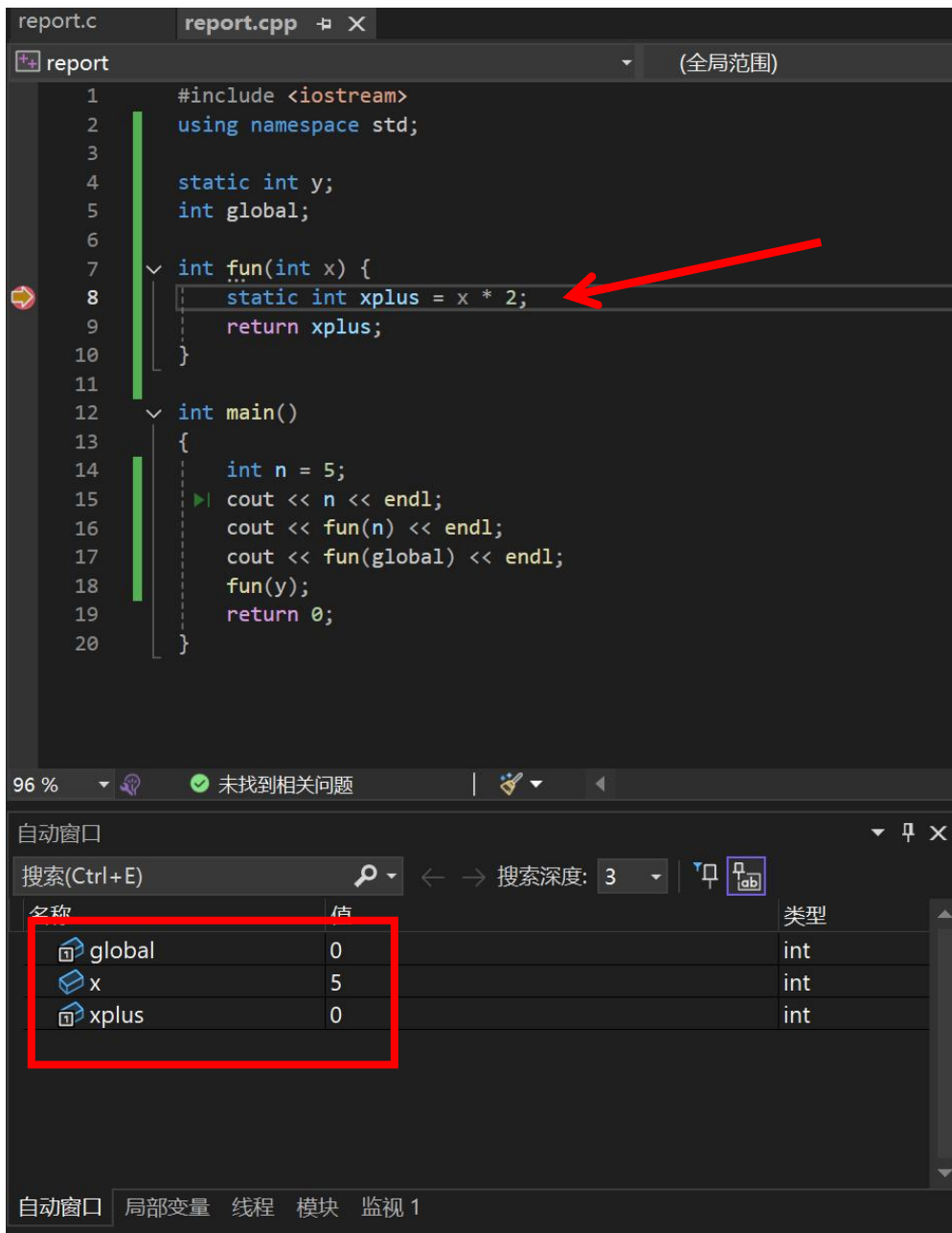
当调试停在函数内部的断点处时，自动窗口和局部变量窗口会显示当前函数的形参和所有已声明的自动变量及其值

也可以在监视窗口中手动添加想要观察的形参或自动变量名
通过单步执行（F10 或 F11），观察这些窗口中变量值的变化

形参(x)：在函数开始执行时创建，在函数结束时销亡。

自动变量(xplus)：在函数内部或代码块内部声明的未使用static修饰的普通局部变量

它们在进入其函数 {} 作用域时创建，在退出作用域时销亡。



```
report.c report.cpp X (全局范围)
1 #include <iostream>
2 using namespace std;
3
4 static int y;
5 int global;
6
7 int fun(int x) {
8     static int xplus = x * 2;
9     return xplus;
10 }
11
12 int main()
13 {
14     int n = 5;
15     cout << n << endl;
16     cout << fun(n) << endl;
17     cout << fun(global) << endl;
18     fun(y);
19     return 0;
20 }
```

96 % 未找到相关问题

自动窗口

名称	值	类型
global	0	int
x	5	int
xplus	0	int

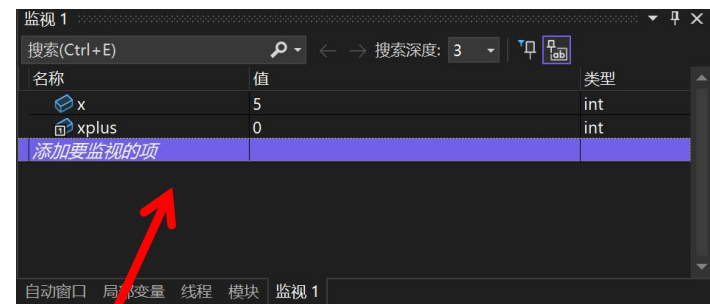
自动窗口 局部变量 线程 模块 监视 1

2 查看静态局部变量的变化情况

当调试停在函数内部的断点处时，自动窗口和局部变量窗口也会显示当前函数的静态局部变量

可以在监视窗口中手动添加想要观察的变量名

通过单步执行（F10 或 F11），观察这些窗口中变量值的变化



```
1  #include <iostream>
2  using namespace std;
3
4  static int y;
5  int global;
6
7  int fun(int x) {
8      static int xplus = x * 2;
9      return xplus;
10 }
11
12 int main()
13 {
14     int n = 5;
15     cout << n << endl;
16     cout << fun(n) << endl;
17     cout << fun(global) << endl;
18     fun(y);
19     return 0;
20 }
```

96 % 未找到相关问题

自动窗口

搜索(Ctrl+F) 搜索深度: 3

名称	值	类型
fun 将返回	10	int
x	0	int
xplus	10	int

2 查看静态局部变量的变化情况

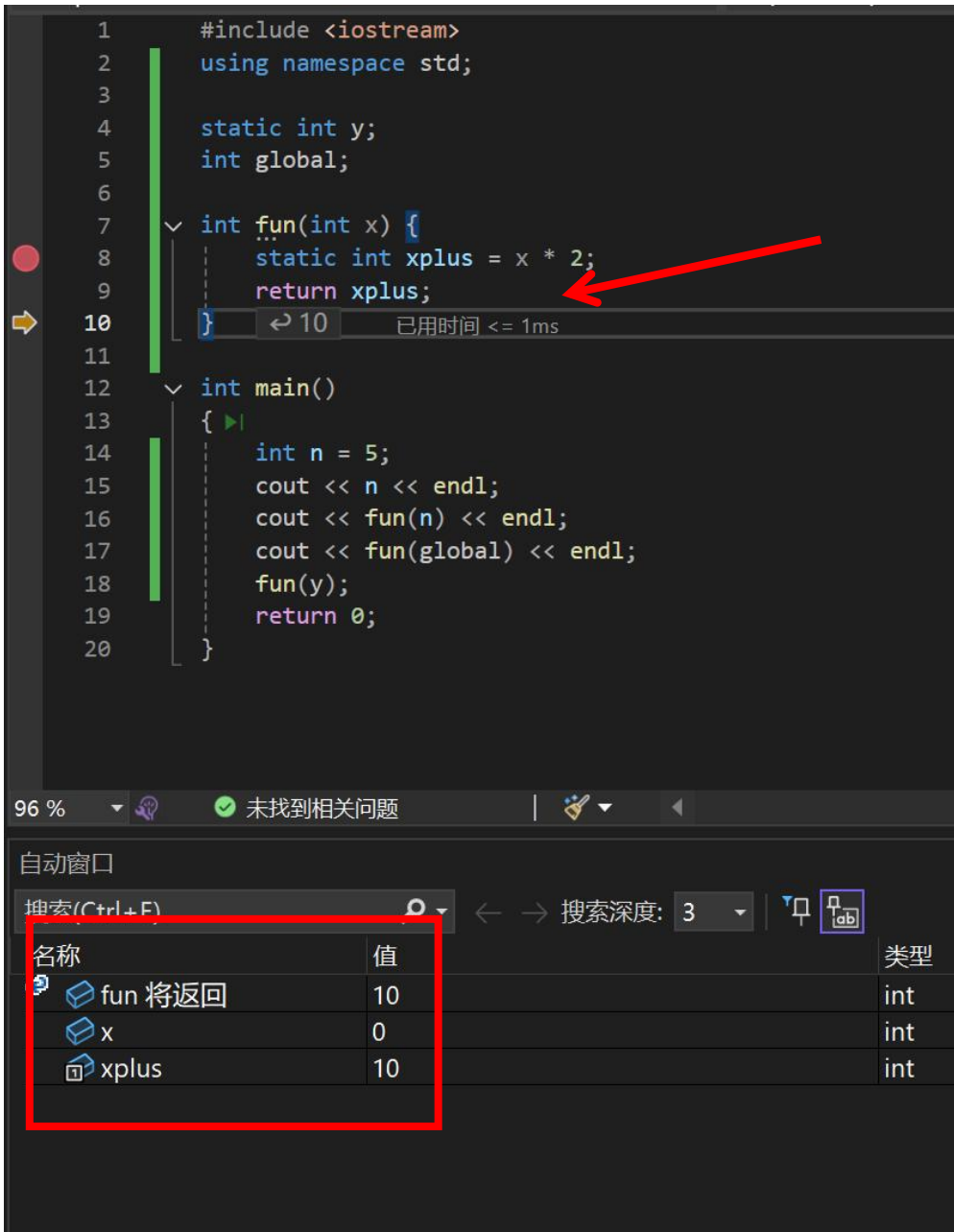
当调试停在函数内部的断点处时，自动窗口和局部变量窗口也会显示当前函数的静态局部变量

可以在监视窗口中手动添加想要观察的变量名

通过单步执行（F10 或 F11），观察这些窗口中变量值的变化

静态局部变量(xplus)在程序第一次执行到其声明语句时初始化，并且在整个程序运行期间都存在，即使函数调用结束，它们的值也会保留到下一次函数调用。

静态局部变量的作用域仍然是局部的，即只能在声明它们的函数内部访问。



2 查看静态局部变量的变化情况

当调试停在函数内部的断点处时，自动窗口和局部变量窗口也会显示当前函数的静态局部变量

可以在监视窗口中手动添加想要观察的变量名

通过单步执行（F10 或 F11），观察这些窗口中变量值的变化

本程序中输出为：

```
C:\homework\Debug\report.exe
5
10
10
```

这是因为静态局部变量只能初始化一次

后续再次进入fun函数的初始化无法改变xplus的值

反汇编 report.c report.cpp

report (全局范围)

```
1 #include <iostream>
2 using namespace std;
3
4 static int y;
5 int global;
6
7 int fun(int x) {
8     static int xplus;
9     xplus = x * 2;
10    return xplus;
11 }
12
13 int main()
14 {
15     int n = 5;
16     cout << n << endl;
17     cout << fun(n) << endl;
18     cout << fun(global) << endl;
19     fun(y);
20     return 0;
21 }
```

96 % 未找到相关问题

自动窗口

搜索(Ctrl+E) 搜索深度: 3

名称	值	类型
fun 将返回	10	int
x	5	int
xplus	10	int

自动窗口 局部变量 线程 模块 监视 1

2 查看静态局部变量的变化情况

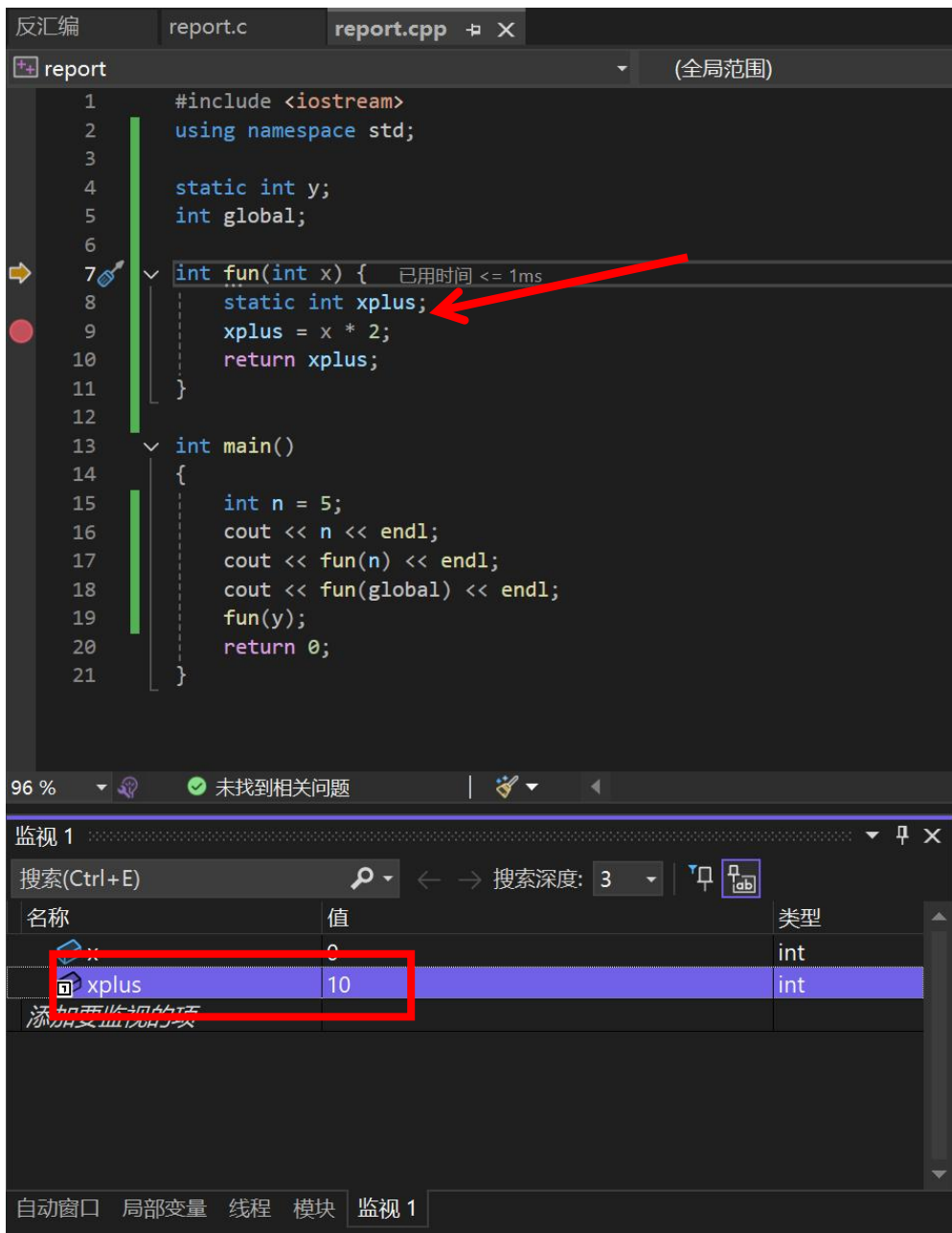
当调试停在函数内部的断点处时，自动窗口和局部变量窗口也会显示当前函数的静态局部变量

可以在监视窗口中手动添加想要观察的变量名

通过单步执行（F10 或 F11），观察这些窗口中变量值的变化

改变fun函数，xplus初始化为0

xplus第一次在函数内被赋值为10，返回10



2 查看静态局部变量的变化情况

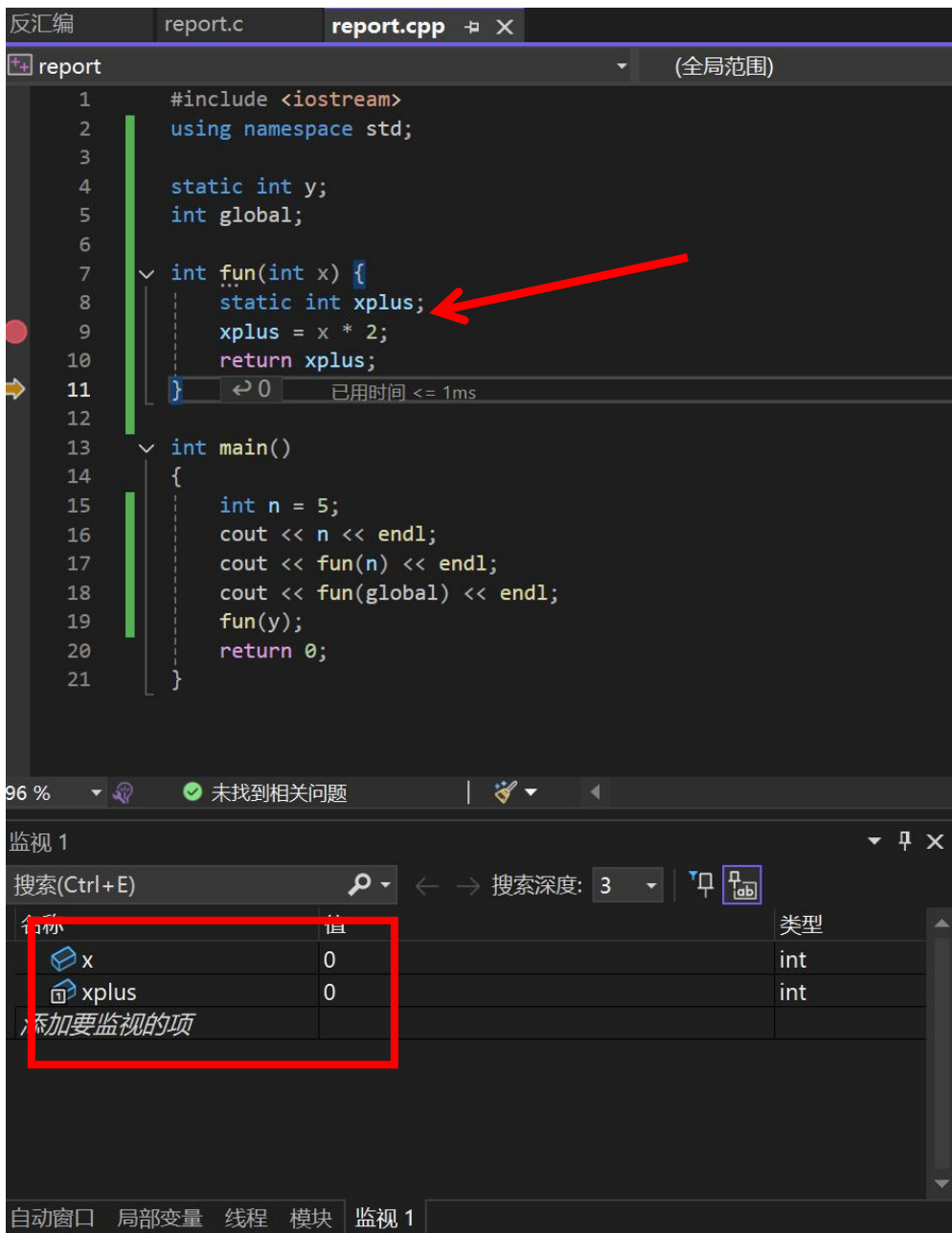
当调试停在函数内部的断点处时，自动窗口和局部变量窗口也会显示当前函数的静态局部变量

可以在监视窗口中手动添加想要观察的变量名

通过单步执行（F10 或 F11），观察这些窗口中变量值的变化

第二次进入fun函数时，可以观察到xplus值为10

由前面的结论，xplus不会再被初始化为0



2 查看静态局部变量的变化情况

当调试停在函数内部的断点处时，自动窗口和局部变量窗口也会显示当前函数的静态局部变量

可以在监视窗口中手动添加想要观察的变量名

通过单步执行（F10 或 F11），观察这些窗口中变量值的变化

xplus被赋值为0，返回0

2.2

report.cpp

report

```
1 #include <iostream>
2 using namespace std;
3
4 static int x = 30;
5 extern int global;
6 extern int modify();
7
8 int fun(int t) {
9     return t+3;
10 }
11
12 int main()
13 {
14     cout << modify() << endl;
15     x = 20;
16     cout << fun(global) << endl;
17     cout << fun(x) << endl;
18     return 0;
19 }
```

96 % 未找到相关问题

report-sub.cpp

report

```
1 #include <iostream>
2 using namespace std;
3
4 static int x = 10;
5 int global;
6
7 int modify() {
8     return x * 2;
9 }
```

96 % 未找到相关问题

监视 1

搜索(Ctrl+E)

名称	值	类型
global	0	int
x	30	int

输出

显示输出

```
"repor
"repor
"repor
"repor
```

2 查看两种全局变量测试程序如图(带sub)

1. 在进入modify函数前，main函数中有全局变量x(静态)与global(外部)，值分别为30/0

2. 按F11进入自定义函数modify后，视监到x值为10，符合预期，这是因为两个x存储在不同位置，这两个不同的变量只是变量名“恰好完全相同”

global

0

x

10

添加要监视的项

6

7

8

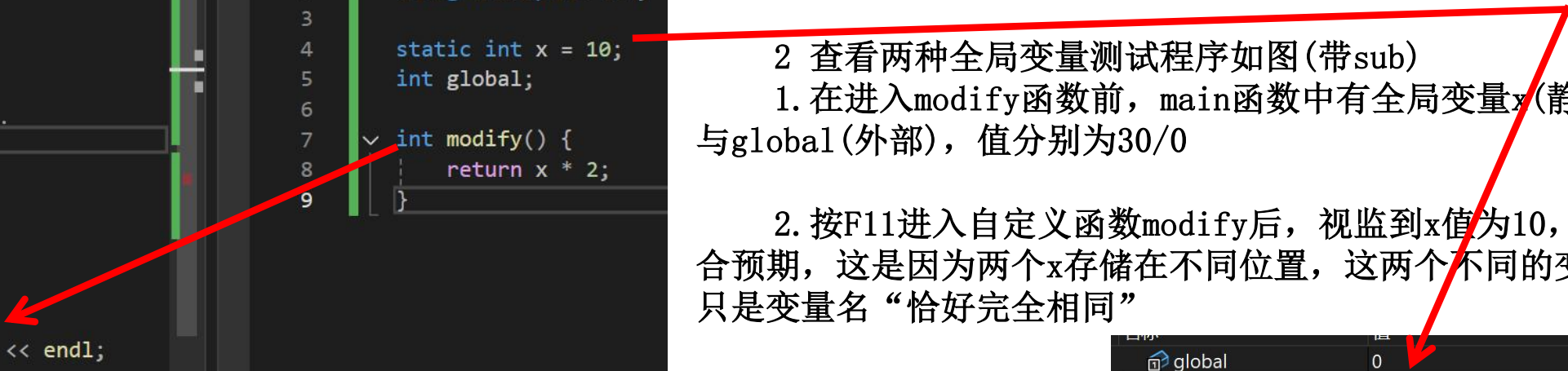
9

int modify() {

return x * 2;

}

已用时间 <= 1ms



report.cpp

report

(全局范围)

```
1 #include <iostream>
2 using namespace std;
3
4 static int x = 30;
5 extern int global;
6 extern int modify();
7
8 int fun(int t) {
9     return t+3;
10 }
11
12 int main()
13 {
14     cout << modify() << endl;
15     x = 20;
16     cout << fun(global) << endl;
17     cout << fun(x) << endl;
18     return 0;
19 }
```

report-sub.cpp

report

```
1 #include <iostream>
2 using namespace std;
3
4 static int x = 10;
5 int global;
6
7 int modify() {
8     return x * 2;
9 }
```

监视 1

搜索(Ctrl+E)

搜索深度: 3

lab

名称	值	类型
global	0	int
x	30	int

输出

显示输出

“repor

“repor

“repor

“repor

2 查看两种全局变量测试程序如图(带sub)

1. 在进入modify函数前, main函数中有全局变量x(静态)与global(外部), 值分别为30/0

2. 按F11进入自定义函数modify后, 视监到x值为10, 符合预期, 这是因为两个x存储在不同位置, 这两个不同的变量只是变量名“恰好完全相同”

3. 若将modify改为修改global, 返回2, 可以看到在跳出modify后global仍被正确赋值为2

```
7 int modify() {
8     global = 2;
9     return global;
10 }
```

2.3/2.4

13 {

cout << modify() << endl;

15 x = 20; 已用时间 <= 1ms

16 cout << fun(global) << endl;

17 cout << fun(x) << endl;

18 return 0;

19 }

监视 1

搜索(Ctrl+E)

搜索深度: 3

lab

名称	值	类型
global	2	int
x	30	int

添加要监视的项

查看不同类型变量值：
可以在监视中直接手动添加所要监视的变量的名字
也可以在局部变量/自动窗口中查看

但无论如何都需要及时打好断点
(根据需要在变量初始化前/后)
或者直接使用F10/F11从头开始单步调试

一维数组在这里也可以查看首地址

二维数组则查看内部一维数组的首地址

3.1 开始

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int n = 2025;
7     float deci = (float)5.19;
8     char ch = 'x';
9     bool flag = true;
10
11     int arr[] = { 2025,6,18,23,5959};
12     int mat[][2] = { {192,168},{174,220},{24,252},{980,101} };
13
14
15     return 0;
16 }
```

监视 1

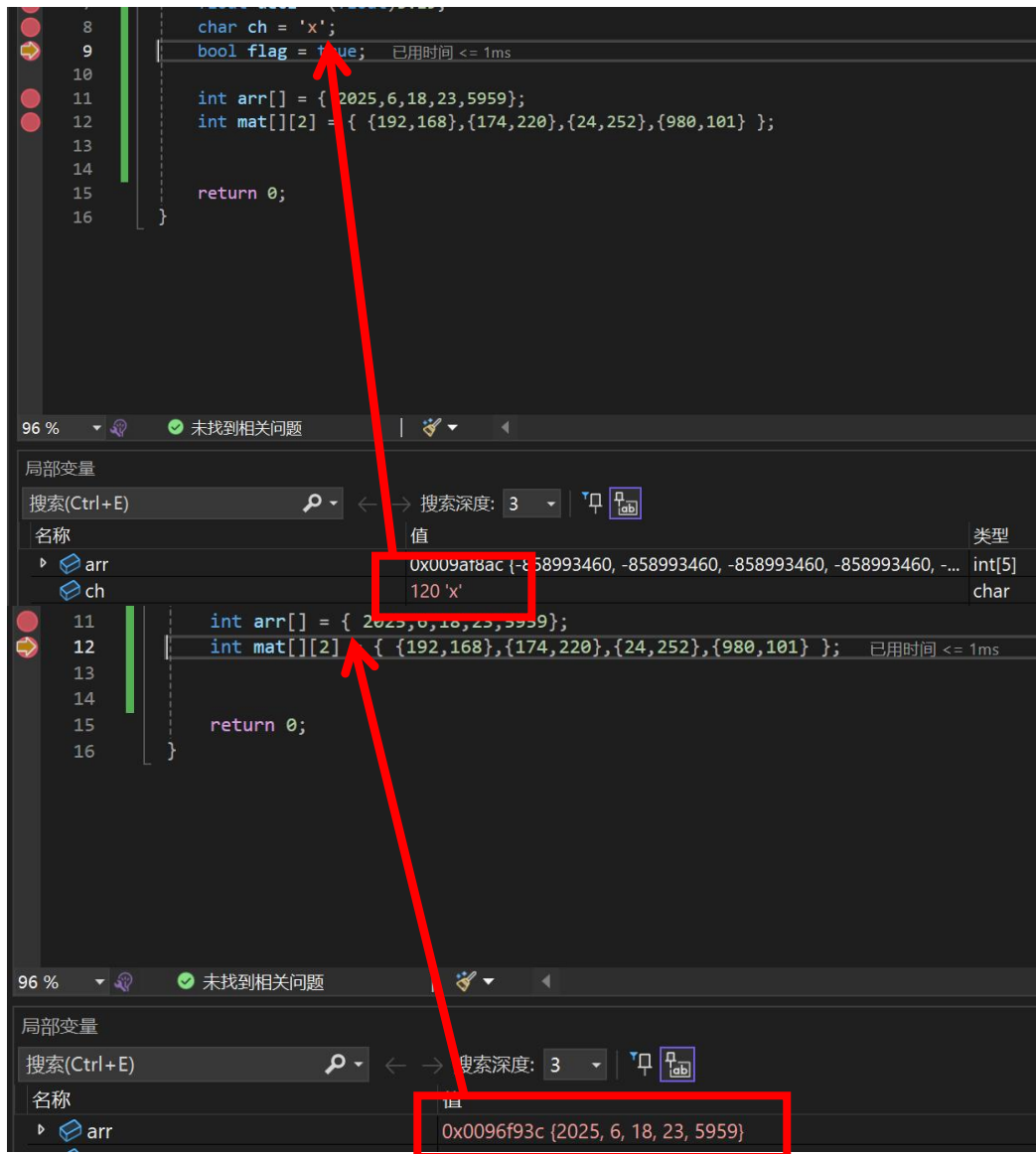
搜索(Ctrl+E) 🔍 搜索深度: 3

名称	值	类型
n	-858993460	int
deci	-107374176.	float
ch	-52 '?'	char
flag	true (204)	bool
arr	0x0096f93c {-858993460, -858993460, -858993460, -858993460, -...}	int[5]
mat	0x0096f914 {0x0096f914 {-858993460, -858993460}, 0x0096f91c {-...}	int[4][2]

添加要监视的项

1 2 3

自动窗口 局部变量 线程 模块 监视 1



查看不同类型变量值:

可以在监视中直接手动添加所要监视的变量的名字
也可以在局部变量/自动窗口中查看

但无论如何都需要及时打好断点

(根据需要在变量初始化前/后)

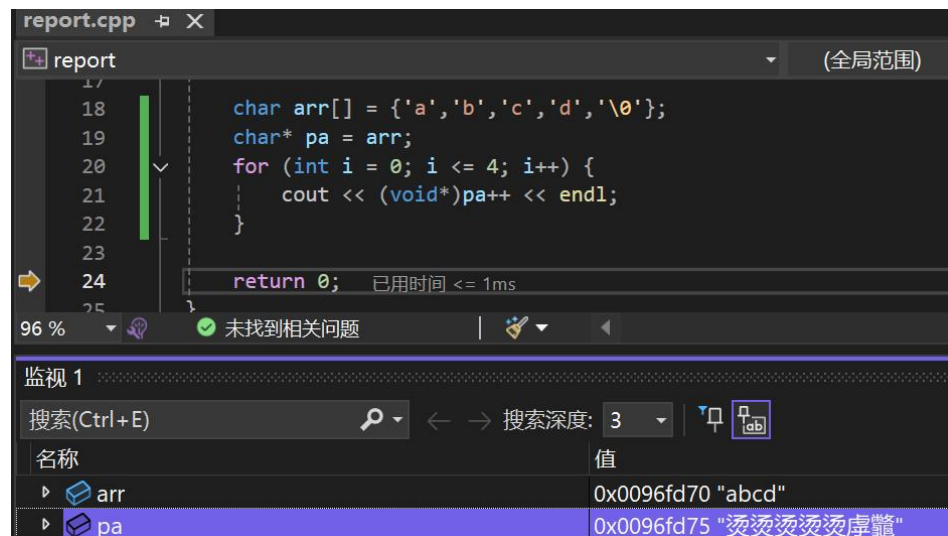
或者直接使用F10/F11从头开始单步调试

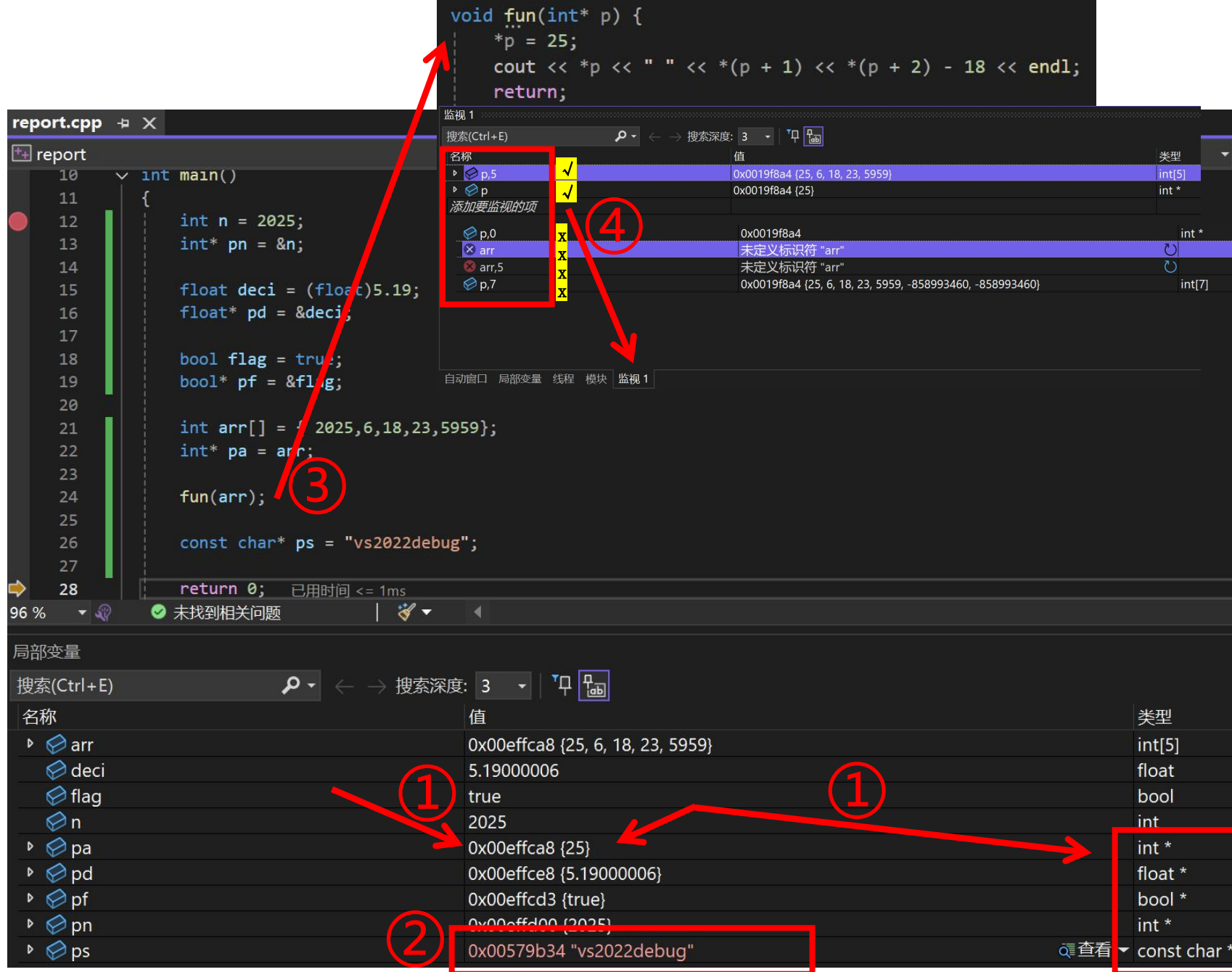
在每执行一个语句后，发生变化的变量会自动变为红色

如果发生指针越界访问

以更明显的字符数组为例，在执行完循环输出地址后，p已经越界指向了分配以外的空间，此时如果以串方式输出则会越界读，如果修改*p则会越界写，int等类型数组同理。

3.9





以上一个程序稍作修改为例 ①
指针变量在 值 一栏显示的就是这个变量存储的地址；在相邻 {} 中可以查看这个地址中元素/数组的值；

在右端类型可清楚看到不同的基类型；

可以发现的是，对于无名字符串常量，②
可以通过指针读取它的地址。同时这串地址中的值也能够正确显示出来。

对于实参是一维数组名，形参是指针的情况，在监视窗口中查看形参p会显示它指向的值(p[0])。③

要查看整个/部分原数组的内容，④
在监视窗口中输入“p, size”。

其中p是传入的形参名 size是我们关心的元素个数。如本例输入 p, 5 就会显示从p指向的地址开始的5个整数。

较为合理的size是(0, maxSize]。

maxSize为原数组大小。如果输入原数组名(如arr)由于局部函数屏蔽，不会显示出原数组的内部元素信息 (如图)。

从3.1-3.7的内容

```
report.cpp
report
(全局范围)
4 int main()
5 {
6     int n = 2024;
7     int* pn = &n;
8     int& m = n;
9     int* pm = &m;
10    cout << m << " " << n << endl;
11    cout << m++ << " " << n << endl;
12
13    int arr[] = { 2,4,5,2,0,9,8 };
14    int* pa = arr;
15    int (&b)[7] = arr;
16    int* pb = b;
17
18    return 0; 已用时间 <= 1ms
```

局部变量

名称	值	类型
arr	0x00affaac {2, 4, 5, 2, 0, 9, 8}	int[7]
b	0x00affaac {2, 4, 5, 2, 0, 9, 8}	int[7] &
m	2025	int &
n	2025	int
pa	0x00affaac {2}	int *
pb	0x00affaac {2}	int *
pm	0x00affaf4 {2025}	int *
pn	0x00affaf4 {2025}	int *

C:\homework\Debug\report.exe

```
2024 2024
2024 2025
```

指针变量在 值 一栏显示的就是这个变量存储的地址；在相邻 {} 中可以查看这个地址中元素/数组的值；在右端类型可清楚看到不同的基类型。

引用相比指针则有本质上的区别。因为引用相当于变量的一个别名，从pn pm存储的地址完全相同可以看出这一点。

显然根据前置知识指针变量的存储需要占用空间而编译器的内部却会进行优化引用使得引用不占用额外的存储空间

在初始化方面，与指针不同的是，引用必须在声明时进行初始化(尽管指针不进行初始化也会导致野指针)，并且后续无法解除这种 绑定 关系。

3.8结束

3.9在前面已完成